EnGN: A High-Throughput and Energy-Efficient Accelerator for Large Graph Neural Networks

Lei He[†,*]
Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China[†],
University of Chinese Academy of Sciences[*]

***Abstract*—Inspired by the great success of convolutional neural networks on structural data like videos and images, graph neural network (GNN) emerges as a powerful approach to process non-euclidean data structures and has been proved powerful in various application domains such as social network, e-commerce, and knowledge graph. However, such graph data maintained in IT companies can be extremely large and sparse, thus employing GNNs to deal with them requires substantial computational power and memory bandwidth, which induces the considerable energy and resources cost spent on general-purpose CPUs and GPUs. In addition, GNN operating on irregular graphs can hardly be fitted to the conventional neural network accelerators or graph processors, which are not designed to support the computation abstraction of GNNs.**

This work presents a specialized accelerator architecture, EnGN, to enable high-throughput and energy-efficient processing of large-scale graph neural networks. The proposed EnGN is designed to accelerate the three key stages of GNN propagation, which is abstracted as common computing patterns shared by typical GNNs. To support the key stages simultaneously, we propose the ring-edge-reduce(RER) dataflow that tames the poor locality of sparsely-and-randomly connected vertices, and the RER PE-array to practice RER dataflow. In addition, we utilize a graph tiling strategy to fit large graphs into EnGN and make the best use of the hierarchical on-chip buffers through adaptive computation reordering and tile scheduling. The experiments on representative GNN models with the input of realistic graphs show that EnGN achieves performance speedup by 1802.9X and 19.75X and energy efficiency by 1326.35X and 304.43X on average compared to the CPU and GPU baselines empowered by the state-of-the-art software frameworks, respectively.

I. Introduction

Recently, the success of deep learning methods in many fields has provoked a keen interest in generalizing neural network architectures to non-Euclidean data, such as manifolds and graphs. However, traditional deep neural networks, such as convolutional neural network (CNN) [29], long short term memory (LSTM) [22], are proposed to work for regular grid-like structures in Euclidean space, they are not trivially portable to non-Euclidean data domains like graphs. Therefore, graph neural networks (GNNs) are recently emerging as a powerful approach for graph processing and achieved unparalleled performance on many classic graph processing tasks, such as citation network [42], [46], social networks [9], and knowledge graph [19], [47]. The success of graph neural network propelled the deployment of GNNs to the real-world production system. For example, Alibaba's AliGraph [55] and Euler [1] platform leverage GNNs to analyze the e-commerce graph data of billion users and items.

Unfortunately, large graph-based neural network has gone beyond what existing deep learning frameworks and graph processing frameworks are designed for [32]. Particularly, high-throughput and low-latency inference is highly demanded in many applications using GNNs. For instance, recommendation system in Taobao [30] that leverages GNNs to mine billion-scale e-commerce data typically needs to perform real-time recommendations to millions of customers shopping at the same time. Thereby, high performance GNN processing frameworks, such as Deep Graph Library (DGL) [2], Pytorch Geometric (PyG) [15], and Neugraph [32] are becoming prevalent. However, due to the overhead of the massive memory parallelism, processing and update activities incurred by large graphs, these GNN software frameworks generally adopt large compute nodes equipped with multiple CPUs or GPUs to deal with large-scale graph, which results in high cost and energy overhead. For example, NeuGraph uses eight GPUs to handle a dataset with million vertexes [32]. Therefore, the potentials of GNNs performance and energy efficiency are still bounded by the hardware architectures assumed by these frameworks.

Intuitively, a specialized GNN architecture is a promising option to improve the efficiency of GNN processing together with the GNN frameworks. How to forge such a general and efficient architecture for diverse GNN models is from the popular graph convolutional networks to graph recurrent networks is a non-trivial task.

GNN algorithms fuse the merits of neural network and the concept of iterative graph propagation. However, neither state-of-the-art neural network processors [12] nor the graph processing accelerators [18] are suitable hardware for GNN processing. First of all, the traditional deep learning accelerators (DLA) are designed to support the convolution or matrix multiplication operations that extract features from regular data structure such as image or audio, but they do not support the other two critical processing stages of GNN propagation. The aggregate stage gathers neighbor features using edges on the graph, which requires not only the ability to process edge information, but also involves frequent but irregular scalar operations and random memory accesses induced by the random traversal of the large but sparse graph.

There are also many graph processing accelerators [23], [25], [45], [49] designed for traditional graph algorithms, such as PageRank, Breadth-First-Search, etc. However, GNNs can hardly be mapped to the existing graph processors. Traditional graph processors [49] are able to support the aggregate and

update stages of graph propagation model, but they generally do not work well for the feature extraction stage of GNN propagation, because they mostly support simple arithmetic operations as required in the traditional graph algorithms. Prior works of graph processors do not show the capability of processing the high-dimension and dynamic vertexes property or the tensors of learned neural parameters in current GNN algorithms, because the dimensions of vertex properties in traditional graph algorithms are usually relatively low and invariant in processing iterations.

Therefore, in order to accelerate practical GNN-based applications that process real-world large-scale graphs, a GNN accelerator has to resolve the obstacles that exist in the real-world GNN algorithms: (1) How to tailor an unified architecture that efficiently supports the diverse GNN models and flows not limited to GCNs. It is observed that the dataflow and the dimension of working-set, e.g., the vertex, dynamically changes in wide ranges during the propagation of different GNN layers, requiring a reconfigurable architecture and interconnects to avoid hardware and memory bandwidth under-utility. (2) large graphs containing millions of vertices pose a significant challenge to the design of energy-efficient and compact GNN accelerators with limited on-chip memory space. Particularly, when massive graphs with million vertices are partitioned into sparsely-connected sub-graphs, there will be intensive random and irregular off-chip memory accesses induced, which leads to poor locality that are hard to harness in the aggregate and update stage. And (3) the power-law distribution [5] creates high-degree but imbalanced connection sparsity in large real-world graphs. Accelerator must be able to deal with the imbalanced sparsity distribution, which leads to processing elements under-utility, poor locality and redundant memory access issues in hardware.

To cope with issues, we propose EnGN, a high-throughput and energy-efficient edge-centric accelerator for large graph neural network processing. First, by observing state-of-the-art GNN processing frameworks such as DGL and PyG, we generalize the architecture of typical GNN algorithms into three key stages: the vertex feature extraction stage, the feature aggregate stage, and the graph update stage. In response to the three key stages abstracted from general GNN frameworks, we support the corresponding computing patterns in EnGN, so that it is a general GNN processor and able to support most of the GNN architectures such as GCN, GRN and etc. In EnGN, a ring-edge-reduce (RER) dataflow and the accompanied hardware architecture of RER processing elements (PEs) arrays are designed to simultaneously conduct the stages of vertex property feature extraction, aggregate and vertex update on GNNs. It is known that aggregating the property and updating the vertices distributed in the large but sparse graphs will lead to poor hardware resources and memory bandwidth utility due to poor data locality of vertexes and edges. However, the proposed RER PEs connected into a ring topology leverages the RER dataflow to make vertex property flow between rows of PEs and performs efficient update operations without randomly accessing the vertices and edges from the memory.

Second, for the feature extraction stage, EnGN constructs a graph property aware dataflow (GPA) that decouples the vertex property and the hardware structure, which makes the GNN mapping to the RER array independent of the vertex dimension. Meanwhile, because the change of vertex-property dimension after the aggregate stage, how to schedule the three stages in GNN layers makes a significant impact on the total computation cost. Thus, GPA can dynamically reorder the graph processing stages to reduce the total computation cost of GNN model on accelerator.

Third, considering the footprint of large-scale graphs, EnGN adopts a graph tiling strategy to process the partitioned sub-graphs with high degree of data reusability. Graph tiling aims to partition a large-scale graph into sub-graphs that fit the on-chip memory and maximize the locality. The tiles are strategically scheduled in EnGN to select either row-oriented or column-oriented processing dataflow to maximally reuse vertices between tiles and reduce the overhead caused by the off-chip memory access.

Finally, due to the power-law distribution and sparsity characteristics of the real-world graphs, the accessing frequency to different vertices may vary in a large scale. For example, on Cora citation graph [40], the access frequency of a high-degree vertex is 100x times than that of a low-degree vertex, which causes access imbalance issue. Thus, EnGN comprises a three-level on-chip memory hierarchy, and the L2 memory is a degree-aware vertex cache (DAVC) to locally cache the high-radix vertices that are densely connected to other vertices in graphs. DAVC reduces considerable memory access cost. In summary, our main contributions are the following:

1) An compact but high-throughput accelerator is designed for large graph neural network, which is implemented based on the edge-centric paradigm and supports various large scale GNNs.
2) We proposed a graph property aware and ring-edge-reduce (RER) dataflow to enable the EnGN to handle a vertex with arbitrary dimension property and high throughput update operations. The on-chip memory hierarchy is designed to be aware of the non-uniform distribution of high-radix and low-radix graph vertexes and employ a specialized memory space management to enhance data locality on the chip.
3) We implement the EnGN accelerator in 45nm process and make comprehensive evaluations and compare the performance, power, energy of EnGN to CPU and GPU baselines empowered by the latest high-performance GNN frameworks. Experimental results show that EnGN outperforms CPU and GPU by up to 1802.9X and 19.75X speedup on average, respectively. EnGN also achieves higher energy efficiency by 1326.35X and 304.43X on average compared to the CPU and GPU baselines.

TABLE I: GNN algorithms on EnGN processing model

Algorithms	Feature extraction	Aggregate	Update
GCN	$h_u^l * V_{degree}^{-1/2}$	$V_{temp}^l = acculumate(Res)$	$ReLu(W^l V_{temp}^l)$
GraghSage-Pool	$ReLu(W_{pool}^l V_u^l + b)$	$V_{temp}^l = max(Res)$	$ReLu(W^l concat(V_{temp}^l, h_v^l))$
R-GCN	$h_{r,u}^l * V_{degree}^{-1/2}$	$V_{r,temp}^l = acculumate(Res)$	$ReLu(\sum_{r\in R} W_r^l V_{r,temp}^l)$
Gated-GCN	$Sigmoid\ (W_H^l h_v^l + W_C^l h_u^l)\ \odot h_u^l$	$V_{temp}^l = acculumate(Res)$	$ReLu\ (W^l\ V_{temp}^l)$
GRN	h_u^l	$V_{temp}^l = acculumate(Res)$	$GRU(h_v^{(l)},\ W^l\ V_{temp}^l)$

II. General GNN processing model

A. Graph neural networks

Unlike convolutional neural networks that mainly deal with Euclidean data like images and videos [32], graph neural networks (GNNs) generalize the conventional neural networks to operate directly on non-Euclidean data especially graph data such as social networks and chemical molecules. It has been proven to be supremely successful on tasks like node classification, graph classification, and link prediction. Motivated by the success of GNNs, various GNN architectures have been proposed recently [8], [10], [24], [27], [44], [53].

Graph convolution network (GCN) generalizes the convolution operation from regular image data to non-structural graph data. It can be used for node classification [28] and chemistry molecules architecture analysis [16]. A typical GCN [28] is presented and formulated in Eq. 1:

$$h^{l+1} = ReLu(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}h^l W^l), h^0 = X \tag{1}$$

Note that A is the adjacency matrix of the graph, $W^{(l)}$ is the weight matrix at layer l, $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$ is essentially the output of the normalized graph Laplacian [28] over A where I_N is the identity matrix and $\tilde{A} = A + I_N$.

GraghSage-Pool is proposed in [20] and used for citation network analysis and protein-protein interaction task. Unlike the GCN models, it leverages the averaging function as an aggregation operator and has the source vertex property (h_v^l) involved when updating output in next iteration. The expression of GraphSage-Pool is defined in Eq. 2.

$$h_v^{l+1} = ReLu(W^l concat(ReLu(W_{pool}^l V_u^l + b)), h_v^l) \tag{2}$$

where $concat(\cdot)$ acts as the function that concatenates a vertex's property with the aggregated property of its neighbor vertices and V_u^l is the source vertex property at layer l.

Relational graph convolutional network (R-RCN) is an extension of GCN and used to handle graphs with different edge types. For instance, the edges can be used to represent different relations and have distinct weights definition of W_r^l [39]. Similar to GCN, hidden representation of entities in the $(l+1)^{th}$ layer in R-GCN can be formulated in Eq. 3:

$$h_i^{l+1} = \sigma(W_0^l h_i^l + \sum_{r\in R}\sum_{j\in N_i^r} \frac{1}{c_{i,r}} W_r^l h_j^l) \tag{3}$$

where N_i^r denotes the set of neighbor indices of node i under relation $r \in R$ and $c_{i,r}$ is a normalization constant. $c_{i,r} = |N_i^r|$ is used in prior entity classification work [39].

Gated graph convolution network (Gated-GCN) is proposed in [14] and utilized for community detection. It borrows the idea from gate recurrent neural networks and constructs a propagation function that receives and processes the property of source vertex and destination vertex simultaneously. The propagation function is depicted in Eq. 4.

$$\begin{aligned} h_v^{(l+1)} &= Relu\ (W^l(\sum_{u\in N(v)} \eta_{uv} \odot h_u^l) \\ \eta_{uv} &= Sigmoid\ (W_H^l h_v^l + W_C^l h_u^l) \end{aligned} \tag{4}$$

where $\odot$ refers to element-wise multiplication, $ReLu(\cdot)$ and $sigmoid(\cdot)$ are typical nonlinear activation functions that have been widely adopted in conventional neural networks [29].

Graph Recurrent network (GRN) is similar to recurrent neural network (RNN), but aims to learn vertex representations [37]. GRN is mostly used in NLP tasks, traffic forecasting, and etc. For example, [31] integrates typical RNN units (Gated recurrent unit) into the propagation function as formulated in Eq. 5 to perform graph algorithm learning tasks.

$$h_v^{(l+1)} = GRU(h_v^{(l)}, \sum_{u\in N(v)} W^l h_u^l) \tag{5}$$

Although GNN algorithms are different in terms of architecture and target applications, we notice that they share common computing patterns. 1) GNNs initially condense vertex property of source vertex with learned parameters to obtain more compact feature representations. 2) Afterwards, GNNs usually gather neighbor properties to embed the information of graph topology to the extracted features. and 3) GNNs usually leverages learned parameters to condense the output features obtained in the aggregate stage making GNN capable to learn and perform more complex tasks. GNN accelerators must be able to support the computation abstractions concluded above, in order to support different GNN architectures efficiently.

Algorithm 1 EnGN processing model

Input: Graph $G = (V, E)$, Vertex property $Prop$ and Tmp_{prop}, layer l, learned parameter $W_{feature}, W_{update}$
Output: Vertex Property $Result$

```
for l ← 1 to l_max do
    for each edge e ∈ Edge do
        tmp ← Feature extraction(Prop[e.src], Prop[e.dst], W_feature)
        Tmp_prop[e.dst] ← Aggregate(Tmp_prop[e.dst], tmp)
    end for
    for each edge e ∈ Edge do
        Prop[e.dst] ← Update(Prop[e.dst], Tmp_prop[e.dst], W_update)
    end for
end for
Result ← Prop
```

B. EnGN processing model

According to the goal of the key stages in a typical GNN, the common computing patterns can be abstracted as *feature extraction*, *aggregate*, and *update*. The *feature extraction* stage condenses the property of each vertex in the graph using a neural network. The *aggregate* stage embeds the graph topology into local vertex property by accumulating each

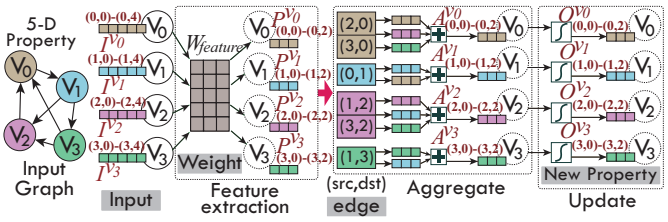

Fig. 1: GCN on EnGN processing model.

vertex's neighbor properties generated in the feature extraction. The choices of aggregate functions include various arithmetic operations such as max, min, and add to produce unified output features. At the end of propagation iteration, the *update* stage leverages learned parameters to further condense the output features obtained in the aggregate stage, then applied a non-linear activation function or GRU/LSTM function to each vertex of the graph before output. Note that when the aggregate stage includes only linear operation, it can be scheduled before or after the feature extraction stage. It also provides an opportunity for EnGN to dynamically adjust the stages of matrix operations to optimize EnGN performance, which will be introduced in section IV. On top of the abstraction, we propose a unified EnGN processing model that can cover general GNN models using the common computing functions as shown in Algorithm 1. Suppose the graph is represented as $G(V, E)$ where V and E represent the set of vertices and edges in the graph respectively. $Property$ is the set of vertex property of the graph. By default, the input graph is stored as a coordinate list (COO). Each edge in the graph is a tuple (src, dst, val), where val usually stands for the edge property and it depends on graph definition. The EnGN execution flow follows the neighborhood aggregation strategy, which iteratively updates the representation of vertices by aggregating representations of their neighbors. Since all the vertices in the graph will be processed in each iteration for GNN algorithms, EnGN is presented as an edge-centric processing model to ensure more efficient memory accesses [54].

For each edge, both the source vertex property and the destination vertex property are condensed with $W_{feature}$ using $feature\_extraction(\cdot)$ to obtain a temporary property tmp. Then tmp is added to the destination property using $aggregate(\cdot)$ function. Since there may be multiple edges that are incident to the same destination vertices, $aggregate(\cdot)$ is essentially a reduce function. When all the destination vertices are reduced, an activation function or the followd user-defined operator with learnable weights W_{update} are used to filter the output using $update(\cdot)$ function.

To help understand the EnGN execution model, we present a vivid example of GCN [28] processed by the EnGN architecture as shown in Fig. 1. Suppose an input graph has four vertices and each vertex has a 5-dimension property. The input property of the vertices are denoted as I^{v_0}, I^{v_1}, I^{v_2}, I^{v_3}. In $feature\_extraction(\cdot)$ function, the feature extraction function takes both the vertex property i.e. I^{v_0}, I^{v_1}, I^{v_2}, I^{v_3} and associated weight matrix $W_{feature}$ as input. Then it has the weight matrix multiplied with the high-dimension input

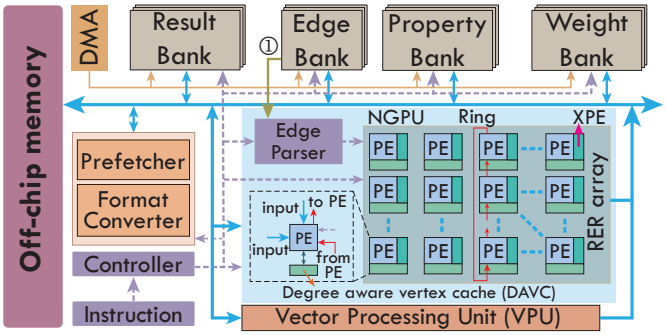

Fig. 2: EnGN hardware architecture.

vertex property to generate low-dimension temp features. Note that the size of the weight matrix is associated with both the input property dimension and output temp feature dimension. In this example, the size of the weight matrix is 5×3. With the feature extraction function, the input vertex properties are converted to 3-dimension temp features donated as P^{v_0}, P^{v_1}, P^{v_2}, P^{v_3}. In $aggregate(\cdot)$ function, it receives the results of $feature\_extraction$ function and aggregates the property of each vertex's incoming neighbors. As shown in Fig. 1, the temp properties of vertex 2 and 3 i.e. P^{v_2}, P^{v_3} are added to temp property of vertex 0 as vertex 2 and 3 are incoming neighbors of vertex 0 P^{v_0} according to the graph topology. When the aggregation stage is done, $update(\cdot)$ starts. It has the vertex features i.e. A^{v_0}, A^{v_1}, A^{v_2}, A^{v_3} filtered using an activation function. The filtered output properties denoted as O^{v_0}, O^{v_1}, O^{v_2}, O^{v_3} become the input to the next iteration.

Similar to the GCNs, we also have the rest of the typical GNN algorithms mentioned in section II mapped to the EnGN processing model. Table I summarizes the resulted EnGN processing functions.

III. EnGN Architecture

A. EnGN hardware architecture

On top of the unified EnGN processing model, we develop a customized EnGN accelerator as shown in Fig. 2. It integrates a neural graph processing unit (NGPU) to perform Feature extraction, Aggregate, and Update operation in a unified architecture. It has an array of homogeneous processing elements (PE) and the array size is 128×16. Each PE unit contains a local register file to store the temporary results and act as intermediate for inter-PE communication. Each PE in the same column of the Ring-Edge-Reduce (RER) array is connected to its neighbors in a ring network to perform aggregate operation and each PE in the same row of the RER array can process a vertex property, which means the NGPU can process 128 vertices simultaneously. However, such processing parallelism requires substantial memory bandwidth. Thereby, to avoid performance degradation, EnGN optimizes the memory access patterns for vertex data and edge data moving. For source vertex data access in the large graph, we adopt the graph tiling technique and ensure that the source vertex fetching only induces accesses to the continuous memory addresses. For random destination vertex accesses in the aggregate and

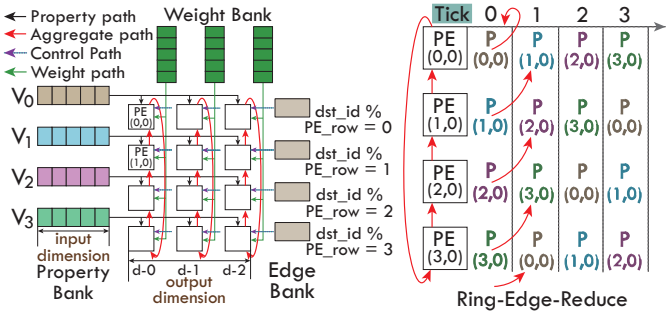

Fig. 3: Architecture details.

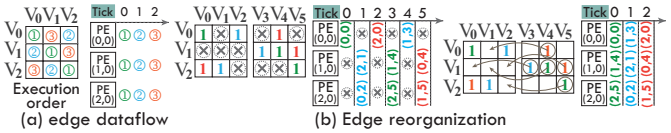

Fig. 4: Edge reorganization.

update stage, EnGN leverages the hashed edge data layout and multi-level cache method to avoid write conflicts and improve data hit rate in the compact on-chip buffer. During processing, the edge parser of NGPU reads the edge list of the graph from the edge banks and parses it into bit-stream that controls the PE-array to perform inter-row aggregate operation (① in Fig. 2). In addition, as shown in Fig. 2, each PE in the NGPU is attached by an XPE to perform activation functions, bias operation, and rounding operation in the update stage. A vector processing unit (VPU) is used to deal with different feature extraction, aggregate and update functions of GNNs illustrated in Table I. Two auxiliary modules: Prefetcher and Format converter, are used to assist the memory accesses and improve the input graph format compatibility respectively.

1) The RER PE array: The feature extraction stage maps the high dimensions property of vertices to the low dimensions by using the learned weight matrix, and this stage is simply matrix multiplication operation. As shown in Fig. 3, in order to handle arbitrary-dimension property of GNN algorithms, we propose the graph property aware (GPA) dataflow to decouple the input property of the vertex and the hardware computing structure. In this manner, each PE in the same column of PE-array is responsible for a single dimension of vertex property and each PE in the same row handles a single vertex. The properties of a vertex are arranged in columns and aligned in the property bank. The dimensions of input vertex property become independent to the hardware architecture and can be continuously injected to the PE-array regardless of the array size and the property dimension. In this manner, the processing unit can handle vertex with arbitrary dimension property.

2) The Ring Edge Reduce topology for PE communication: The aggregate procedure needs to collect the information according to the edge information. Thereby, as shown in Fig. 3, each row of the PE-array in NGPU possesses a dedicated edge bank and each PE in the same row receives the same control signal parsed from edge list in the graph to gather the corresponding vertex property. Meanwhile, because each PE needs to broadcast its own vertex features generated by the feature extraction stage to all other PEs in the same column, aggregating the received information simultaneously can result in a large amount of hardware resource and power consumption. Thereby, inspired by the ring-all-reduce concept [13], we propose the ring-edge-reduce (RER) aggregate dataflow to conduct aggregate stage inside the PE array instead of moving the data out to the buffer. As shown in Fig. 3, because each column of PE performs the same operations without any communication in between, each PE in the same column of array is connected to its neighbors through an on-chip network of ring topology. Each PE in the same column only communicates with its two nearest neighbors (north, south). In our design, the PE sends its data to the northern neighbors and receives the data sent from the southern neighbors for property aggregating. In this manner, a PE can select the relevant vertices to aggregate based on the control signal parsed from the edges during the data flow across the ring.

The RER dataflow makes the hardware design simple yet efficient when the graph is dense and the vertex properties that flow through the ring are mostly used for aggregation. However, many of the large graphs in practice are sparse and aggregation in PEs is inactive in many cases. A RER dataflow example on a sparse graph and the adjacency matrix is shown in Fig. 4(a) and (b). The computing array is assumed to be 3×3. In cycle 0, three edges from different edge banks will be fetched and the properties of V_0, V_1 and V_2 will flow across the ring at the same time. It takes the RER three cycles to complete the movement of the three vertex properties and the corresponding aggregation on V_0, V_1 and V_2. Similarly, it takes the RER another three cycles to repeatedly transfer the three vertex properties through the ring to aggregate on V_3, V_4 and V_5. Thereby, it takes the RER at least 6 cycles to perform the aggregate of the graph and many of the time slots are idle as marked with crosses in the figure.

To improve the efficiency of the aggregation, we further analyze the reason for the idle time slots. For example, PE(1, 0) is idle in Cycle 0 because the edge to be processed is $2 \rightarrow 1$ and it does not have the properties of vertex 2 yet. However, if it fetches the edge $1 \rightarrow 4$ first, it can perform the aggregate of vertex 4 using the property of vertex 1 at Cycle 0. With this observation, we propose to reorganize the edges in each edge bank to ensure the vertex properties flowing through the ring is used as much as possible. The right part of Fig. 4(b) exhibits the reorganized edges and the corresponding aggregation. With the edge reorganization, the aggregate completes in 3 cycles and the computing array is fully utilized. Basically, the order of the vertex properties flowing through the ring is known given the computing array. The required vertex property of each edge is also determined. Thereby, reorganizing the edges in each edge bank based on the order of the vertex properties flowing through the ring can maximize the aggregation efficiency of the computing array.

B. The On-chip Memory Hierarchy

PE register file The register files (RF) equipped in the PEs are divided into four groups including source vertex

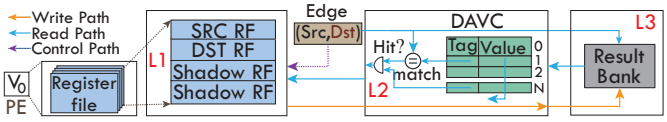

Fig. 5: Memory hierarchy.

groups(SRC RF), destination vertex groups (DST RF), and two shadow groups (Shadow RF), which is depicted in Fig. 5. The SRC RF stores the source vertex values generated in the feature extraction stage. The DST RF stores the destination vertex feature updated during the aggregate and update stages. In addition, there are two programmer-invisible Shadow RFs holding the SRC and DST vertex values previously generated by the PEs of the same column.

Multiple-level caches The real world graph has up to millions of vertices. Although the graph tiling technique adopted by EnGN helps fit the sub-graphs into the on-chip buffer, the set of vertices in the sub-graphs will still outsize the register files of the PE array. Meanwhile, the result banks are used to store the temporary aggregate results. PE frequently accesses the long-latency result bank will result in performance degradation. Consequently, as shown in Fig. 5, we insert a degree aware vertex cache (DAVC) between the result banks and the register file of each PE to improve the performance of the EnGN. The register file, DAVC, and the result banks are regarded as the first, second, and last level memories on chip, respectively. All capacity of DAVC is used to cache high-degree vertices. The reason will be illustrated in section V. DAVC uses the destination vertex id of edges as the line tag to determine whether the access to the vertex data hit or not in the DAVC. If hit, the vertex data will be directly read to DST RF in the PE unit. Otherwise, EnGN will access the last-level result banks. In this manner, the DAVC can alleviate the overhead incurred by the result bank accesses.

C. EnGN dataflow

Fig. 6 illustrates the execution flow of a GNN layer on the proposed EnGN hardware based on GCN model [28]. Similar to the EnGN processing model, the execution stage of one GNN layer on EnGN is also divided into three stages: feature extraction, aggregate, and update stage. The feature extraction and aggregate stage are executed in pipeline while the update stage can only be triggered when the aggregate stage is completed. Without losing generality, as shown in Fig. 3, we consider a small design that consists of 4×3 PEs and the input graph is the same as the Fig. 1. For brevity, we only describe the flow at the $PE(0,0)$ and $PE(1,0)$.

Feature extraction: At cycle 0, $PE(0,0)$ receives the first dimension of the input vertex V_0 property $I(0,0)$ at layer $l-1$, and the weight data $W(0,0)$ to generate the temporary property result $P(0,0)$ and store result in local register file.

At cycle 1, $PE(0,0)$ receives the second dimensions of input vertex V_0 property $I(0,1)$ and the weight data $W(1,0)$. Both of them are used to update the temporary result $P(0,0)$.

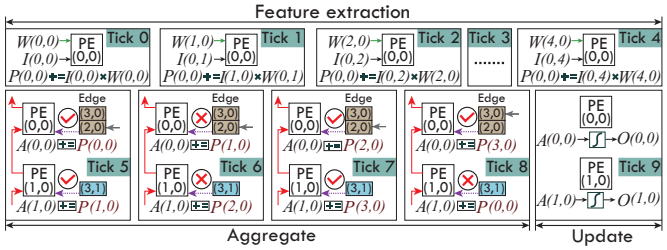

Fig. 6: EnGN Dataflow.

On the follow-up cycles, the operations on $PE(0,0)$ are similar to cycle 1. The following new input properties $I(0,2)$, $I(0,3)$, $I(0,4)$ and the associated weight data $W(2,0)$, $W(3,0)$, $W(4,0)$ continuously feed into $PE(0,0)$ until the dimensions of vertices have been covered.

At cycle 4, the result $P(0,0)$ is generated and stored into the register file, waiting to be used by the aggregate stage.

Aggregate: At cycle 5, Since the edge buffer contains vertex 0, $PE(0,0)$ uses $P(0,0)$ to generate the aggregate result $A(0,0)$. At the same time, $PE(0,0)$ sends $P(0,0)$ to $PE(3,0)$ and prepares for receiving the $P(1,0)$ sent from $PE(1,0)$.

At cycle 6, $PE(0,0)$ receives $P(1,0)$ from $PE(1,0)$, $P(1,0)$ will not be utilized to update $A(0,0)$ because vertex 1 does not exist in the according edge list provided by the edge buffer, which means vertex 1 is not connected to vertex 0. Meanwhile, $PE(0,0)$ send $P(1,0)$ to $PE(3,0)$ and prepares for receiving $P(2,0)$ sent from $PE(1,0)$.

At cycle 7, $PE(0,0)$ receives $P(2,0)$ from $PE(1,0)$ and leverages it to update the aggregate result $A(0,0)$ because vertex 2 is in the according edge list provided by the edge buffer. Meanwhile, $P(2,0)$ is sent to $PE(3,0)$.

At cycle 8, $PE(0,0)$ receives $P(3,0)$ from $PE(1,0)$ and leverages it to update the aggregate result $A(0,0)$. In this case, all vertices have been traversed by each PE. Thus, $A(0,0)$ summarizes all the relevant edge information and will be stored in the register file to wait for the update stage.

Update: At cycle 9, since all edges associated with vertex 0 has been traversed, the aggregate result $A(0,0)$ will not be changed. Thereby, the update stage is triggered, and the aggregate result $A(0,0)$ is processed by the user-defined apply function, i.e. activation function $ReLu(\cdot)$ in GCNs, to generate the result $O(0,0)$ as the input to the next layer l.

IV. ENGN OPTIMIZATION

A. Observations of GNN computing

To further optimize the EnGN design, we try to explore the characteristics of GNN algorithms and seek key observations that may guide the EnGN architecture optimization. Suppose the input graph $G=(V,E)$ with N vertices and E edges is depicted with an adjacency matrix $A\in\mathbb{R}^{N\times N}$. The vertex property of the graph is $X\in\mathbb{R}^{N\times F}$ with F channels and the learned filters i.e. weight is $W\in\mathbb{R}^{F\times H}$ where H is output property dimension. Then, the output of the GNN i.e. O can be represented as Eq. 6:

$$O=\sigma(A(XW)) \tag{6}$$

According to the formulation of GNNs, we obtain three major exploitable observations:

1. *The dimension of the vertex property in GNN generally exhibits serious variation across the iterations compared to traditional graph algorithms.*

The dimension of the vertex property is determined by both the application (i.e. the number of vertices in the graph) and the architecture of the GNN model (i.e. the condensed feature dimension) according to Eq. 6. While GNN is an iterative algorithm, the output feature in current iteration becomes input feature to the next iteration. Thereby, the vertex property dimension mostly relies on the weight array size i.e. the architecture of the GNN model and it varies across the different iterations. In each iteration, vertex property dimension may either increase or decrease after the computing.

2. *The order of feature extraction processing and aggregate processing in GNNs are exchangeable when the operator in aggregate processing is sum.*

When the operator used in *aggregate* is *sum* which is widely adopted in GNN algorithms, the computing in Eq. 6 can be changed to Eq. 7 without affecting the result because of matrix multiplication associative law. While the amount of operations using the distinct computing order is also different, we may choose the order that incurs less computation in each iteration.

$$O = \sigma((AX)W) \tag{7}$$

3. *The weight size of GNNs is independent to the size of the input graph and it is usually small. While the input graphs can be large and typically dominate the memory accesses.*

According to Eq. 6, the weight size of GNNs is irrelevant to the number of vertices in the graph. In this case, the weight size can be much smaller compared to the graphs that may include millions of vertices, which is also a key distinction from conventional neural networks. Input graphs will dominate the memory accesses and dealing with the large graphs in GNNs will be critical to the accelerator performance.

B. Dimension-aware stage re-ordering

According to Observation 1 and 2, the processing order of GNN stages, the feature extraction, aggregate and update stages, will not affect the computing results, but it can change the total number of operations in a GNN. We analyze the quantity of operations when using different computing order, and aim to find the best way to schedule the stages. For *feature_extraction*, the number of operations i.e. multiply-accumulate in Eq. 6 and Eq. 7 are the same and it is equal to $N \times F \times H$. Similarly, *update* does not change with the computing order. Nevertheless, for *aggregate*, the order of GNN computing leads to different number of operations i.e. accumulation in *aggregate*. When Eq. 6 is used, the number of operations is $E \times F$. When Eq. 7 is chosen, the amount of operations becomes $E \times H$.

While the property dimension varies as observed in last subsection, F is not equal to H. To reduce the total computing, when the input vertex property dimension F is larger than output feature dimension H, we should choose Eq. 6 for GNN computing. Otherwise, we should use Eq. 7. Following this idea, we propose a dimension-aware stage reordering (DASR) strategy based on the input and output property dimension comparison. The DASR can be implemented by altering the instruction sequence that defines the computing order of GNNs, so it will not incur additional hardware overhead.

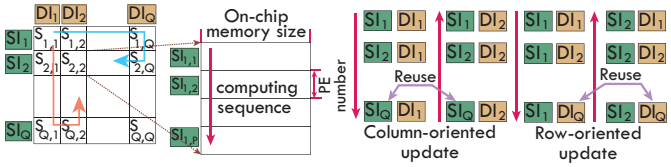

Fig. 7: Graph tiling and tile scheduling

C. Graph tiling and scheduling

According to Observation 3, a real-world graph that can be very large dominates the memory accesses in GNNs and it cannot be fitted to the limited on-chip memory of EnGN. To address this issue, EnGN tiles the large graph into intervals and shards using a grid partition approach proposed in [57]. The basic idea of the grid partition is to divide all the vertices into Q disjointed intervals. Then the edges of the graph with both source and destination vertices limited to one interval can be partitioned into Q^2 disjointed shards. Each shard must be fitted to the on-chip memory of EnGN accelerator to ensure efficient computing without external memory accesses.

With the tiling, EnGN processes with the granularity of a tile. For each tile, the number of vertices remains larger than the row size of the PE array while each row of PE can only handle a single vertex at one time according to the dataflow proposed in prior section. In this case, the vertices are processed in batch and the batch size is equal to the row size of the PE array. The batch processing of a tile is described in Fig. 7. Instead of conducting *feature_extraction* and *aggregate* sequentially, we have them overlapped. Basically, *aggregate* starts when a batch of vertices complete *feature_extraction*.

Although tiling ensures EnGN to process using just the data that are accommodated in the on-chip buffers, there are still data dependency between the different tiles. The order of the tile execution essentially affects the data reuse and the amount of external memory accesses accordingly. Thereby, tile scheduling is also an important design option that needs to be intensively optimized.

The graph is split into a 2D array of tiles. The tiles in each row have the same source vertices while the tiles in the same column have the same destination vertices. Intuitively, we may schedule in either a row manner or a column manner. In the column-major order, new source vertices must be reloaded tile by tile while the the destination vertices in the same interval reside in on-chip buffer until the column of tiles complete execution. In the row-major order, source vertex properties can be buffered until the whole row of tiles are processed. We also notice that there are also shared data between neighboring columns or rows and propose to schedule with an S-shape as shown in Fig. 7. For example, the bottom tile of a column

TABLE II: I/O cost

	Read Size	Write Size
Column-oriented	$(Q^2-Q+1)F+QH$	QH
Row-oriented	$QF+(Q^2-Q+1)H$	Q^2H

share the same source vertices with the bottom tile in next column. Similar data sharing can be observed in row manner.

The different tile scheduling strategies mainly differ on the external memory accesses and we quantitatively analyze the I/O cost. For column-major order, each column of tiles requires to load Q tiles of source vertices and the total amount of load is Q^2. When neighboring column data reuse is considered, the amount of data to be loaded becomes Q^2-Q+1. While the destination vertices in each column can be reused, the total amount of write is Q. For row-major order, the amount of read is the same, but the amount of write is much larger, because tiles in a row generates many intermediate output and must be frequently swapped to external memory among different tile execution. The total amount of write is Q^2. While the dimension of the vertex property also affects the amount of I/O cost and the dimension of input vertex property and output vertex property is usually different according to Observation 1, we further take the vertex property dimension into consideration and the I/O cost is summarized in Table II.

Suppose that the latency of read and write external memory is equal. Comparing the overhead of the two different tile scheduling strategies, we obtain the following formulation:

$$IO_{column-major} - IO_{row-major} \approx (Q-1)(2H-F) > 0 \quad (8)$$

Based on Eq. 8, it can be concluded that the column-major order scheduling outperforms the row-major order scheduling when F is smaller than 2H. Otherwise, row-major order scheduling is preferred. While F and $2H$ are mostly determined by the GNNs and the comparison varies, we employ an adaptive scheduling to minimize the external memory accesses. The adaptive scheduling option is explicitly encoded in the instructions which are generated at compilation time based on the GNN models.

V. EVALUATION

A. Experimental setup

Accelerator simulator We built a cycle-accurate simulator to measure the performance of EnGN accelerator. This simulator models each module of EnGN accelerator faithfully and the timing behaviors of the modules are co-verified with the synthesized RTL design. The simulator is also integrated with DRAMSim [36] to characterize the off-chip memory accesses.

EnGN configuration&implementation EnGN includes a 512KB multi-bank property buffer, a 512KB multi-bank weight buffer, a 256KB multi-bank edge buffer, a 256KB multi-bank result buffer and a 64KB distributed vertex cache. With the design parameters, we synthesized the EnGN accelerator using Synopsys Design Compiler (DC) with the TSMC 45nm process technology, conducted the placing-and-routing using Synopsys ICC compiler (ICC), and estimated the power consumption using Synopsys PrimeTime (PT).

TABLE III: GNN models and Datasets.

Model	Graph	#Vertices	#Edges	#Feature/ #Relation	Label
GCN	Cora (CA) [40]	2708	10556	1433	7
	PubMed (PB) [40]	19717	88651	500	3
	Nell (NE) [6]	65755	251550	5415	210
GS-Pool	CoraFull (CF) [4]	19793	126842	8710	67
	Reddit (RD) [20]	232965	114.6M	602	41
	Enwiki (EN) [3]	3.6M	276.0M	300	12
	Amazon (AN) [34]	8.6M	231.6M	96	22
Gated-GCN	Synthetic A (SA) [7]	4.19M	67.1M	100	16
	Synthetic B (SB) [7]	8.38M	134.2M	100	16
GRN	Synthetic C (SC) [7]	12.41M	205.3M	64	16
	Synthetic D (SD) [7]	16.76M	268.4M	50	16
R-GCN	AIFB (AF) [39]	8285	29043	91	4
	MUTAG (MG) [39]	23644	192098	47	2
	BGS (BG) [39]	333845	2166243	207	2
	AM (AM) [39]	1666764	13643406	267	11

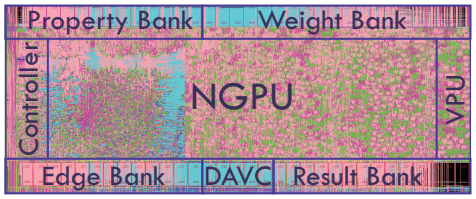

Fig. 8: Layout (45nm).

Baselines We compared the performance and energy efficiency of EnGN with two baseline computing platforms including a **CPU** platform equipped with Intel Xeon(Skylake) 6151@3.0GHz processor and 696GB DRAM and a **GPU** baseline platform equipped with NVIDIA Tesla V100 SXM2 and 32GB HBM2. To achieve the best performance of the baseline platforms, we utilized two state-of-the-art GNN frameworks i.e. DGL [2] and Pytorch geometric (PyG) to execute the GNN algorithms. The implementations are denoted as CPU-DGL, CPU-PyG, GPU-DGL, and GPU-PyG respectively.

GNN models and datasets To benchmark the performance of EnGN accelerator, we implemented a set of typical GNN models on two distinct groups of datasets as shown in Table III. The top part includes four algorithms i.e. GCN [28], GraphSage-Pool (GS-Pool) [20], Gated-GCN [14], and GRN [50], which are mainly used for semi-supervised classification. The four algorithms performs on seven real-world graph datasets and four synthetic graph datasets. The bottom part mainly targets at knowledge graph application and R-GCN [39] is a widely adopted entity classification algorithm. The corresponding datasets are from four typical knowledge graphs. Particularly, note that the feature and label columns represent the dimension of a vertex and the number of labeled classes respectively.

Evaluation Metrics In this experiment, we take the end-to-end inference time of GNNs as the performance metric, billion operations per second (GOP/s) as the throughput metric and billion operations per second per Watt (GOPS/W) as the energy-efficiency metric.

B. Experimental results

Layout&Area Fig. 8 shows the physical layout of EnGN, and the total area of EnGN is $74.45mm^2$. The estimated peak power consumption is $7.1W$ when operated at 1GHz.

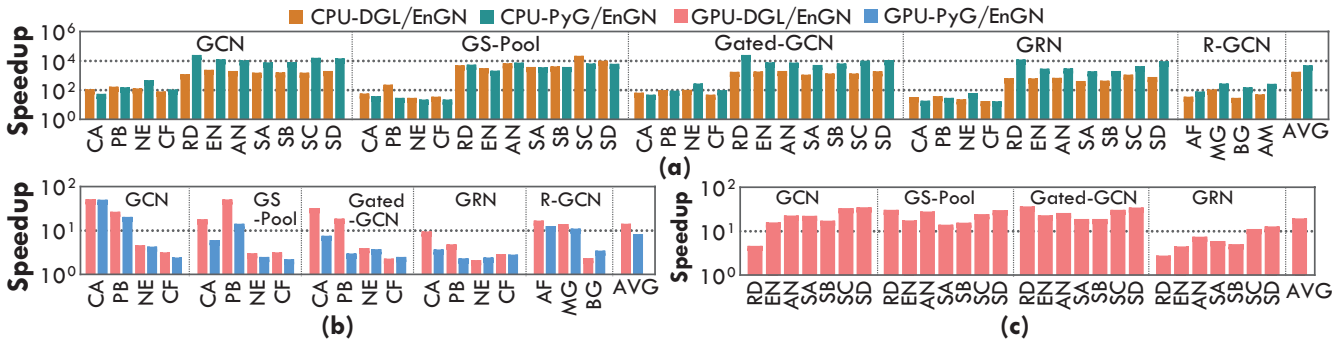

Fig. 9: Performance comparison of EnGN over CPU and GPU. (a) Performance speedup of EnGN over CPU-DGL and CPU-PyG. (b) Performance speedup of EnGN over GPU-DGL and GPU-PyG on small datasets. (c) Performance speedup of EnGN over GPU-DGL on large datasets. Since GPU-PyG runs out of memory, it is omitted.

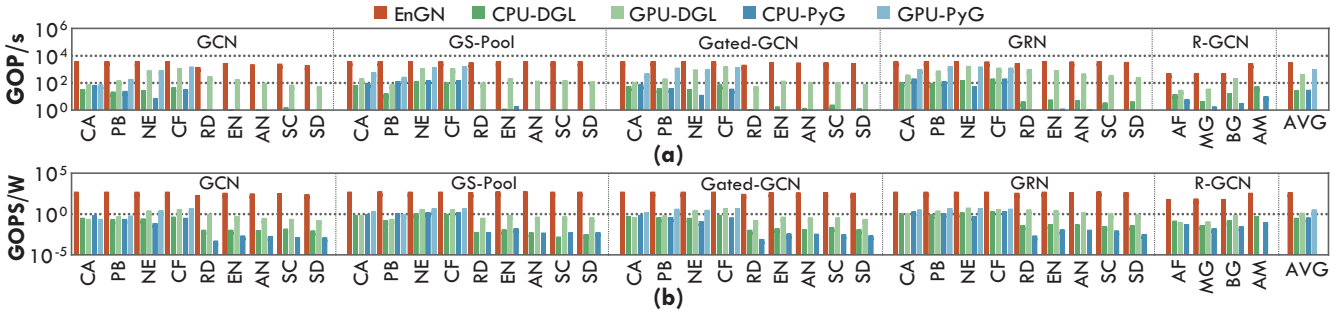

Fig. 10: Throughput and energy efficiency of EnGN, CPU and GPU. (a) Throughput (b) Energy efficiency

Performance We compare the performance of EnGN to that obtained from the baseline computing platforms including CPU-DGL, GPU-DGL, CPU-PyG, and GPU-PyG. The comparison result is shown in Fig. 9. The average performance speedup of all the models on all the datasets over CPU-DGL and CPU-PyG are 1802.9X and 5108.4X respectively as shown in the last bar of Fig. 9(a) denoted as AVG. Also it can be observed that EnGN outperforms CPU in all cases despite the software frameworks, datasets and GNN models. We also compare EnGN with GPU using DGL and PyG respectively. However, PyG runs out of memory on larger datasets due to the lack of sufficient memory optimizations. Thus, we only compare GPU-DGL on large graph datasets as shown in Fig. 9 (c). On small graph datasets, we have both GPU-DGL and GPU-PyG compared and the comparison is presented in Fig. 9(b). EnGN gains 14.41X and 8.35X performance speedup over the GPU-DGL and GPU-PyG respectively on the small datasets. On large datasets, EnGN achieves 19.75X speedup on average. In general, although GPU performs much better than CPU, EnGN still outperforms in all cases.

On top of the computing platforms, we further compare the performance speedup of EnGN on different datasets, it can be noticed that EnGN typically shows significantly higher performance speedup when the dimension of the graph feature is small. For instance, the performance speedup of GS-Pool on SD with smaller feature dimension is around 10613.17X on CPU-DGL and 35.34X on GPU-DGL while the performance speedup of GS-Pool on CF with the larger feature dimension is less than 36.47X on CPU-DGL and less 2.22X on GPU-DGL. While EnGN with fine-grained dataflow can make good use of the computing resources, the computing efficiency does not vary much with the datasets, which will be illustrated in the following experiments. In contrast, CPU and GPU prefers datasets with high-dimension features that can be accessed sequentially and efficiently. Thereby, the different graph features of the datasets lead to distinct performance speedup. Meanwhile, we also find that the performance speedup of EnGN on RD with relatively high-dimension feature is actually clearly higher than the average performance speedup. The reason for this exception is that RD has rather high average degree than the other graphs. The high-degree graph requires large memory footprint during aggregation stage and can no longer be fitted to the on-chip memory or cache. Thereby, the computing efficiency degrades. **Throughput** Fig. 10(a) shows the measured throughput of EnGN, CPU and GPU on the GNN benchmark in Table III. The average throughput of EnGN is 3265.87 GOP/s which achieves 79.7% of the peak throughput i.e. 4096GOP/s. In contrast, the measured average throughput of CPU-DGL and CPU-PyG is only 29.29 GOP/s and 31.95 GOP/s respectively, which is 111.50X and 102.21X lower. GPU with massive parallel processing units performs much better. The average throughput using GPU-DGL and GPU-PyG is 426.30 GOP/s and 1056.91 GOP/s respectively. Still, the throughput of EnGN is 7.66X and 3.09X higher. To gain further insight of the computing efficiency on different GNN models and datasets, we measure the computing efficiency of the different computing architectures including EnGN, CPU and GPU. As shown in Fig. 10(a), the computing efficiency of EnGN typically keeps steady and does not vary much with the models and datasets while CPU and GPU are more sensitive

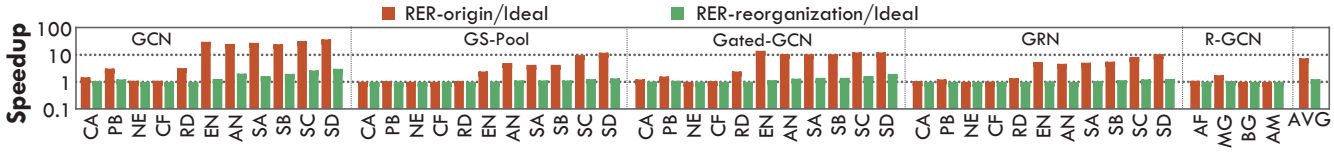

Fig. 11: Performance comparison of GNNs with original edge layout and reorganized edge layout. Note that both the performance is normalized to the ideal performance with optimal computing resource utilization.

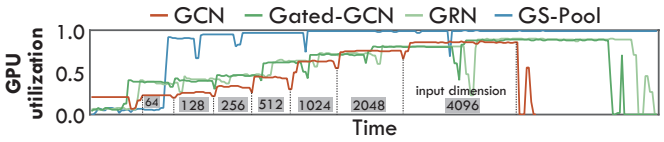

Fig. 12: GPU utilization w.r.t feature dimensions.

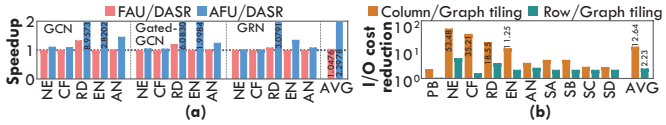

Fig. 13: (a) Speedup of DASR over FAU and AFU and (b) I/O cost reduction of Graph tiling over Column and Row.

and the computing efficiency usually fluctuates with the feature dimension of the graphs as pointed out in prior section.

Energy Efficiency To obtain the energy efficiency of the different computing architectures, we need to measure its power first. The power consumption of CPU and GPU is obtained from the power meter and NVPROF respectively. The power consumption of EnGN is estimated using PrimeTime. The power consumption of CPU, GPU and EnGN is 150W, 300W and 7.1W respectively. On top of the power consumption, we further calculated the energy efficiency using the total amount of operations and the execution time. The energy efficiency is shown in Fig. 10(b). The average energy efficiency of EnGN is 1326.35X and 1196.04X higher than CPU-DGL and CPU-PyG respectively. When compared to GPU, the energy efficiency of EnGN over GPU-DGL and GPU-PyG is 213.61X and 133.17X higher on small datasets. The speedup goes up to 529.13X for large datasets on which only DGL can be applied. The great energy efficiency speedup is mainly attributed to the much lower power consumption of the customized EnGN accelerator over the power-hungry general purposed processors and the much higher performance reported in the performance paragraph. The reasons for the higher performance and lower power consumption are already discussed, and we will not dwell on it.

C. EnGN optimization evaluation

Edge reorganization and RER In order to avoid the PE idling in RER, we propose to reorganize the edge list to improve the utilization of the computing array in EnGN. Fig. 11 exhibits the performance comparison of GNNs with reorganized edges and original edges. It can be noted that the edge reorganization approach improves the performance significantly and the average performance speedup is 5.4X. Meanwhile, we find that the proposed edge reorganization approach typically works much better for large datasets. The variation of the benefits is mainly caused by the different proportion of aggregation in the total amount of the GNN computing. While the aggregation in GNNs dominates the computing when the graph is large, thus the performance improvement is higher. In addition, we have the performance normalized to that of an ideal design which utilizes a fully connected PE column. With the fully connected topology, the aggregation can achieve the optimal performance despite the edge organization. When compared to the ideal topology, the proposed RER topology in combination with the edge reorganization approach achieves near optimal performance in various cases. In contrast, the hardware design overhead is much smaller compared to the fully connected topology.

Sensitivity to the variation of vertex dimension The vertex property dimension varies dramatically in GNNs, so to be insensitive to the vertex property dimension variation is of vital importance to a general GNN accelerator design. In this experiment, we generated a synthetic graph with 65000 vertices, 2.5M edges, and 16 classes. Then we change the input vertex dimension from 64 to 4096 gradually to evaluate the computing efficiency variation under the different vertex property dimension setups. We compare the computing variation of EnGN and GPU-DGL. Fig. 12 depicts GPU utilization is lower than 50% when the vertex property dimension is smaller than 512. Moreover, it drops considerably when GPU threads are wasted under some odd vertex dimension setups. In contrast, the PE utilization of EnGN is irrelevant to input vertex property dimension because the dataflow in EnGN decouples the input vertex property dimension and the computing array.

Dimension aware stage re-ordering As mentioned in section IV, the proposed dimension aware stage reordering technique can reduce the total computing cost. In this evaluation, we get rid of the GS-Pool model because its aggregate stage adopts the average operator which hinders the stage reordering. We compared the performance speedup of EnGN that adopts **d**imension-**a**ware **s**tage **r**e-ordering (DASR) strategy to two fixed processing strategy: (1) feature_extraction, aggregate and update (FAU), and (2) aggregate, feature_extraction and update (AFU). Fig. 13(a) illustrates that DASR strategy can improve the performance of EnGN by 1.047x and 2.297x on averages compared to FAU and AFU, respectively. The reason for the poor performance improvement compared to FAU is the output dimensions of GNN models on most datasets are decreasing, which makes no scheduling necessary. However, in reddit datasets, our DASR strategy can improve the performance of EnGN by 1.34x and 8.96x compared to FAU and AFU strategy. This is because the output dimensions of vertex property on the last layer are 210 (Table III), which is higher than that of on the first layer. When the feature extraction stage performs

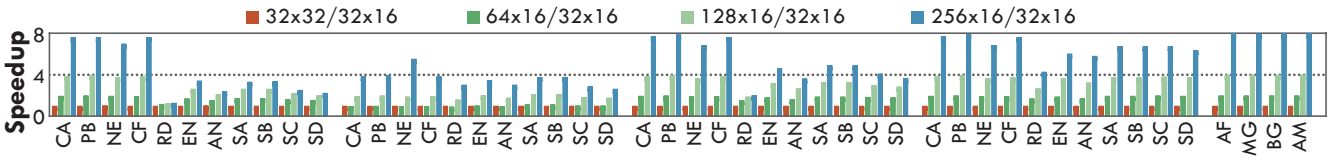

Fig. 14: Performance over number of PEs.

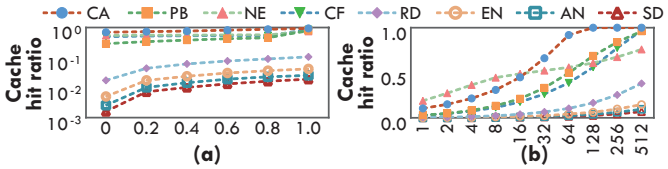

Fig. 15: Cache hit ratio over different proportions (a) and size (KB)(b).

after the aggregate stage, higher dimensions incurs massive accumulate operators in the aggregate stage. In contrast, when we performs the feature extraction stage before the aggregate stage, the dimension will be compressed to 16 and accumulates operators is only 16 for a vertex in the aggregate stage.

Graph tiling scheduling In this evaluation, we leveraged the column-major (Column) and row-major (Row) update strategy as baselines to evaluate our scheduling strategy on GCN model. Fig. 13(b) illustrates the total I/O cost reduction induced by the EnGN scheduling strategy compared to the Column and Row strategies, respectively. In PubMed and large datasets, our graph tiling scheduling strategy only reduces total I/O cost by 3.26x and 1.90x compared to Column strategy. This is because PubMed and large dataset only contains $3 \sim 16$ class labels, which is less than the output dimension of the first layer. In contrast, Nell, Cora-full, and Reddit contains 210, 67, and 41 classes respectively. Thereby, in this case, graph tiling scheduling can reduce the total memory access cost by 29.62x and 3.02x on average when compared to Column and Row, respectively. This is because the Column and Row strategy stick to the fixed policy to update the graph while our graph tiling scheduling can adjust the update dataflow from the Row to Column according to the dimension changes in GNNs.

Degree Aware Vertex Cache (DAVC) DAVC is a standard cache supporting replacement policy like LRU in general. To improve the cache hit rate, we take the vertex degree information into consideration and reserve part of the cache entries for high-degree vertices which are determined with offline static analysis and will not be replaced during the execution. To determine the proportion of the reserved cache entries, we analyze the cache hit rate under various proportion setups ranging from 0 to 1. The experiment in Fig. 15(a) reveals that cache hit rate increases monotonically with the proportion especially for the larger graphs. The main reason is that on-chip cache is too small relative to the large graphs and thus suffers frequent replacement when LRU policy is applied. Thereby, we have all the cache used for high-degree vertices. Meanwhile, we also analyze the influence of cache size on the cache hit rate. Similar conclusion can be drawn as shown in Fig. 15(b). Basically, the cache hit rate for large graphs remains rather low and larger cache size is preferred.

D. Scalability Analysis

Performance over number of PEs Since each row of PE array handles one vertex and each column is in charge of one dimensions of output property, as the input graph and output property dimensions get larger, the system can be scaled up by adjusting the size of PE-array. We varied the size of PE-array in EnGN, where the EnGN with 32×16 PE-array is set as baseline. Fig. 14 show EnGN achieves good scalability on all GNN models and datasets. With the increase of the row number in PE-array, the throughput of EnGN is increasing. However, 32×32 array exhibits no improvement over the baseline. This is because the output property dimensions of the first layer (16) on all models are below the column number of PE array (32), which causes underutilization of PE array. Thereby, we can adjust the size of PE array according to the datasets and the complexity of GNN models to maximize the throughput of EnGN. Fig. 14 also witnessed the performance speedup on large datasets is lower than on small datasets. This is due to the large data has higher edge-to-vertex ratio compared to small datasets, which makes the aggregated stage new bottleneck in large PE arrays.

VI. RELATED WORK

A. GNNs software framework

There is a large amount of work that aims at building an efficient system for graph applications on single node-machines (CPUs) [25], [33], [52], [57], distributed systems [35], [43], [45], [51], and GPUs [17], [26]. However, these graph processing frameworks aim at traditional algorithms, and there is a lack of support for graph neural network computation. Even though TuX2 [56] aims to bridge the gap between graph and traditional machine learning algorithm, it is still unable to support the inference and training stage of emerging GNN algorithms. Thereby, NeuGraph [32] is proposed to recast the graph specific optimization as dataflow optimization based on Tensorflow. Meanwhile, [43] published a geometric learning library for deep learning on irregularly structured input data based on Pytorch. The deep graph library [2] provides a fast implementation of GNN models based on pytorch and MxNet.

NeuGraph, Pytoch geometric, and DGL are generally running on the power-hungry CPU and GPUs, which incurs energy-efficient issues and high cost. More importantly, GPUs suffer from the under-utility of stream processors during parallel GNN computation because of the impact of the irregular graph data structure, which makes energy-efficient issues more serious. Thereby, to address these issues, we build an EnGN accelerator designed for large graph neural network to support energy-efficient graph neural network processing.

B. Deep learning & Graph accelerator

The resurgence of deep neural network (DNN) and its substantial progress in various applications including image, video, and speech spurs the flourishing of the DNN hardware accelerator [38], [41]. For example, Diannao [52] maps DNN onto an array of multiply-add units and employ data tiling policy to exploiting the locality in the parameters. EIE [21] performs inference using compressed technique and accelerates the inherent modified sparse matrix-vector multiplication. Eyeriss [11] is a low power real-time DNN accelerator that exploits zero valued neurons by using run length coding for memory compression. However, these DNN accelerators are designed for traditional DNN such as convolution neural network, which cannot handle GNNs because they lack graph propagation model on the accelerator.

The wide gap between the general-purpose architectures and the unique features of graph processing promotes the rapid development of graph processing-specific accelerators based on FPGA and ASIC. For example, Graphicionado [18] and [48] presented a domain-specific accelerator for graph analytics based on well-defined, popular vertex programming model. However, traditional graph accelerators are designed for traditional graph algorithms, it lacks the computation abstraction required by the neural network, such as tensor and activation operations.

VII. Conclusions

In this paper, we present a high-throughput and energy efficient accelerator EnGN specialized for large graph neural network processing. In order to provide high throughput processing ability and solve the arbitrary dimension change issues in the GNN algorithms, we proposed ring-edge-reduce update dataflow and the accompanied hardware architecture of RER PE-arrays are designed to simultaneously conduct high-throughput processing in the feature-extraction, aggregate and update stages on GNNs. Meanwhile, the proposed graph tiling and scheduling technique cooperating with well-designed three-level memory hierarchy enable EnGN to process large graph efficiently. Experimental results show that EnGN achieves a performance gains of 1802.9X and 19.75X and an energy efficiency of 1326.35X and 304.43X compared to CPUs and GPUs on average, respectively.

References

[1] "A distributed graph deep learning framework. contribute to alibaba/euler development by creating an account on GitHub," original-date: 2019-01-10T06:32:32Z. [Online]. Available: https://github.com/alibaba/euler

[2] "Python package built to ease deep learning on graph, on top of existing DL frameworks.: dmlc/dgl," original-date: 2018-04-20T14:49:09Z. [Online]. Available: https://github.com/dmlc/dgl

[3] "Wikimedia Downloads." [Online]. Available: https://dumps.wikimedia.org/

[4] A. Bojchevski and S. Günnemann, "Deep Gaussian Embedding of Graphs: Unsupervised Inductive Learning via Ranking," arXiv e-prints, p. arXiv:1707.03815, Jul 2017.

[5] A. Broder, R. Kumar, F. Maghoul, P. Raghavan, S. Rajagopalan, R. Stata, A. Tomkins, and J. Wiener, "Graph structure in the web," Comput. Netw., vol. 33, no. 1-6, pp. 309–320, Jun. 2000. [Online]. Available: http://dx.doi.org/10.1016/S1389-1286(00)00083-9

[6] A. Carlson, J. Betteridge, B. Kisiel, B. Settles, E. R. Hruschka, Jr., and T. M. Mitchell, "Toward an architecture for never-ending language learning," in Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence, ser. AAAI'10. AAAI Press, 2010, pp. 1306–1313. [Online]. Available: http://dl.acm.org/citation.cfm?id=2898607.2898816

[7] D. Chakrabarti, Y. Zhan, and C. Faloutsos, "R-mat: A recursive model for graph mining," in SIAM International Conference on Data Mining, 2004. [Online]. Available: http://www.cs.cmu.edu/~christos/PUBLICATIONS/siam04.pdf

[8] J. Chen and J. Zhu, "Stochastic training of graph convolutional networks," 2018. [Online]. Available: https://openreview.net/forum?id=rylejExC-

[9] J. Chen, T. Ma, and C. Xiao, "Fastgcn: Fast learning with graph convolutional networks via importance sampling," CoRR, vol. abs/1801.10247, 2018. [Online]. Available: http://arxiv.org/abs/1801.10247

[10] J. Chen, T. Ma, and C. Xiao, "FastGCN: Fast learning with graph convolutional networks via importance sampling," in International Conference on Learning Representations, 2018. [Online]. Available: https://openreview.net/forum?id=rytstxWAW

[11] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in Proceedings of the 43rd International Symposium on Computer Architecture, ser. ISCA '16. Piscataway, NJ, USA: IEEE Press, 2016, pp. 367–379. [Online]. Available: https://doi.org/10.1109/ISCA.2016.40

[12] Y. Chen, T. Luo, S. Liu, S. Zhang, L. He, J. Wang, L. Li, T. Chen, Z. Xu, N. Sun, and O. Temam, "Dadiannao: A machine-learning supercomputer," in Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture, ser. MICRO-47. Washington, DC, USA: IEEE Computer Society, 2014, pp. 609–622. [Online]. Available: http://dx.doi.org/10.1109/MICRO.2014.58

[13] Z. Cheng and Z. Xu, "Bandwidth reduction using importance weighted pruning on ring allreduce," CoRR, vol. abs/1901.01544, 2019. [Online]. Available: http://arxiv.org/abs/1901.01544

[14] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, "Language modeling with gated convolutional networks," CoRR, vol. abs/1612.08083, 2016. [Online]. Available: http://arxiv.org/abs/1612.08083

[15] M. Fey, "Geometric deep learning extension library for PyTorch: rusty1s/pytorch_geometric," original-date: 2017-10-06T16:03:03Z. [Online]. Available: https://github.com/rusty1s/pytorch_geometric

[16] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," CoRR, vol. abs/1704.01212, 2017. [Online]. Available: http://arxiv.org/abs/1704.01212

[17] J. E. Gonzalez, Y. Low, H. Gu, D. Bickson, and C. Guestrin, "Powergraph: Distributed graph-parallel computation on natural graphs," in Proceedings of the 10th USENIX Conference on Operating Systems Design and Implementation, ser. OSDI'12. Berkeley, CA, USA: USENIX Association, 2012, pp. 17–30. [Online]. Available: http://dl.acm.org/citation.cfm?id=2387880.2387883

[18] T. J. Ham, L. Wu, N. Sundaram, N. Satish, and M. Martonosi, "Graphicionado: A high-performance and energy-efficient accelerator for graph analytics," in The 49th Annual IEEE/ACM International Symposium on Microarchitecture, ser. MICRO-49. Piscataway, NJ, USA: IEEE Press, 2016, pp. 56:1–56:13. [Online]. Available: http://dl.acm.org/citation.cfm?id=3195638.3195707

[19] T. Hamaguchi, H. Oiwa, M. Shimbo, and Y. Matsumoto, "Knowledge transfer for out-of-knowledge-base entities: A graph neural network approach," CoRR, vol. abs/1706.05674, 2017. [Online]. Available: http://arxiv.org/abs/1706.05674

[20] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," CoRR, vol. abs/1706.02216, 2017. [Online]. Available: http://arxiv.org/abs/1706.02216

[21] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, "EIE: efficient inference engine on compressed deep neural network," CoRR, vol. abs/1602.01528, 2016. [Online]. Available: http://arxiv.org/abs/1602.01528

[22] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Comput., vol. 9, no. 8, pp. 1735–1780, Nov. 1997. [Online]. Available: http://dx.doi.org/10.1162/neco.1997.9.8.1735

[23] Y. S. Horawalavithana, "On the Design of an Efficient Hardware Accelerator for Large Scale Graph Analytics," Tech. Rep. [Online]. Available: https://en.wikipedia.org/wiki/Field-programmable

[24] W. Huang, T. Zhang, Y. Rong, and J. Huang, "Adaptive sampling towards fast graph representation learning," in Proceedings of the 32Nd International Conference on Neural Information Processing Systems, ser. NIPS'18. USA: Curran Associates Inc., 2018, pp. 4563–4572. [Online]. Available: http://dl.acm.org/citation.cfm?id=3327345.3327367

[25] S.-W. Jun, A. Wright, S. Zhang, S. Xu, and Arvind, "Grafboost: Using accelerated flash storage for external graph analytics," in Proceedings of the 45th Annual International Symposium on Computer Architecture, ser. ISCA '18. Piscataway, NJ, USA: IEEE Press, 2018, pp. 411–424. [Online]. Available: https://doi.org/10.1109/ISCA.2018.00042

[26] F. Khorasani, K. Vora, R. Gupta, and L. N. Bhuyan, "Cusha: Vertex-centric graph processing on gpus," in Proceedings of the 23rd International Symposium on High-performance Parallel and Distributed Computing, ser. HPDC '14. New York, NY, USA: ACM, 2014, pp. 239–252. [Online]. Available: http://doi.acm.org/10.1145/2600212.2600227

[27] D. Kim, Y. Yoo, J. Kim, S. Lee, and N. Kwak, "Dynamic graph generation network: Generating relational knowledge from diagrams," in 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, June 2018, pp. 4167–4175.

[28] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," CoRR, vol. abs/1609.02907, 2016. [Online]. Available: http://arxiv.org/abs/1609.02907

[29] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," Commun. ACM, vol. 60, no. 6, pp. 84–90, May 2017.

[30] C. Li, K. Jia, D. Shen, C. R. Shi, and H. Yang, "Hierarchical representation learning for bipartite graphs," in Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI-19. International Joint Conferences on Artificial Intelligence Organization, 7 2019, pp. 2873–2879. [Online]. Available: https://doi.org/10.24963/ijcai.2019/398

[31] Y. Li, D. Tarlow, M. Brockschmidt, and R. S. Zemel, "Gated graph sequence neural networks," CoRR, vol. abs/1511.05493, 2016.

[32] L. Ma, Z. Yang, Y. Miao, J. Xue, M. Wu, L. Zhou, and Y. Dai, "Neugraph: Parallel deep neural network computation on large graphs," in 2019 USENIX Annual Technical Conference (USENIX ATC 19). Renton, WA: USENIX Association, Jul. 2019, pp. 443–458. [Online]. Available: https://www.usenix.org/conference/atc19/presentation/ma

[33] G. Malewicz, M. H. Austern, A. J. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski, "Pregel: A system for large-scale graph processing," in Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data, ser. SIGMOD '10. New York, NY, USA: ACM, 2010, pp. 135–146. [Online]. Available: http://doi.acm.org/10.1145/1807167.1807184

[34] J. J. McAuley, C. Targett, Q. Shi, and A. van den Hengel, "Image-based recommendations on styles and substitutes," in SIGIR, 2015, pp. 43–52. [Online]. Available: https://doi.org/10.1145/2766462.2767755

[35] M. M. Ozdal, S. Yesil, T. Kim, A. Ayupov, J. Greth, S. Burns, and O. Ozturk, "Energy efficient architecture for graph analytics accelerators," SIGARCH Comput. Archit. News, vol. 44, no. 3, pp. 166–177, Jun. 2016. [Online]. Available: http://doi.acm.org/10.1145/3007787.3001155

[36] P. Rosenfeld, E. Cooper-Balis, and B. Jacob, "Dramsim2: A cycle accurate memory system simulator," IEEE Comput. Archit. Lett., vol. 10, no. 1, pp. 16–19, Jan. 2011. [Online]. Available: http://dx.doi.org/10.1109/L-CA.2011.4

[37] H. Salehinejad, J. Baarbe, S. Sankar, J. Barfett, E. Colak, and S. Valaee, "Recent advances in recurrent neural networks," CoRR, vol. abs/1801.01078, 2018. [Online]. Available: http://arxiv.org/abs/1801.01078

[38] S. Sarkar, T. Majumder, A. Kalyanaraman, and P. P. Pande, "Hardware accelerators for biocomputing: A survey," in Proceedings of 2010 IEEE International Symposium on Circuits and Systems, May 2010, pp. 3789–3792.

[39] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. vanden Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in The Semantic Web, A. Gangemi, R. Navigli, M.-E. Vidal, P. Hitzler, R. Troncy, L. Hollink, A. Tordai, and M. Alam, Eds. Cham: Springer International Publishing, 2018, pp. 593–607.

[40] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Gallagher, and T. Eliassi-Rad, "Collective classification in network data," Tech. Rep., 2008.

[41] V. Sze, Y.-H. Chen, T.-J. Yang, and J. Emer, "Efficient processing of deep neural networks: A tutorial and survey," Proceedings of the IEEE, 2017.

[42] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in Proceedings of the 36th International Conference on Machine Learning, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. Long Beach, California, USA: PMLR, 09–15 Jun 2019, pp. 6861–6871. [Online]. Available: http://proceedings.mlr.press/v97/wu19e.html

[43] M. Wu, F. Yang, J. Xue, W. Xiao, Y. Miao, L. Wei, H. Lin, Y. Dai, and L. Zhou, "Gram: Scaling graph computation to the trillions," in Proceedings of the Sixth ACM Symposium on Cloud Computing, ser. SoCC '15. New York, NY, USA: ACM, 2015, pp. 408–421. [Online]. Available: http://doi.acm.org/10.1145/2806777.2806849

[44] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," CoRR, vol. abs/1901.00596, 2019. [Online]. Available: http://arxiv.org/abs/1901.00596

[45] C. Xu, C. Wang, G. Lei, Y. Lu, F. Sun, Y. Zhang, X. Li, and X. Zhou, "Omnigraph: A scalable hardware accelerator for graph processing," 09 2017, pp. 623–624.

[46] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" CoRR, vol. abs/1810.00826, 2018. [Online]. Available: http://arxiv.org/abs/1810.00826

[47] K. Xu, L. Wang, M. Yu, Y. Feng, Y. Song, Z. Wang, and D. Yu, "Cross-lingual knowledge graph alignment via graph matching neural network," CoRR, vol. abs/1905.11605, 2019. [Online]. Available: http://arxiv.org/abs/1905.11605

[48] M. Yan, X. Hu, S. Li, A. Basak, H. Li, X. Ma, I. Akgun, Y. Feng, P. Gu, L. Deng, X. Ye, Z. Zhang, D. Fan, and Y. Xie, "Alleviating irregularity in graph analytics acceleration: A hardware/software co-design approach," in Proceedings of the 52Nd Annual IEEE/ACM International Symposium on Microarchitecture, ser. MICRO '52. New York, NY, USA: ACM, 2019, pp. 615–628. [Online]. Available: http://doi.acm.org/10.1145/3352460.3358318

[49] P. Yao, L. Zheng, X. Liao, H. Jin, and B. He, "An efficient graph accelerator with parallel data conflict management," in Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques, ser. PACT '18. New York, NY, USA: ACM, 2018, pp. 8:1–8:12. [Online]. Available: http://doi.acm.org/10.1145/3243176.3243201

[50] J. You, R. Ying, X. Ren, W. L. Hamilton, and J. Leskovec, "Graphrnn: A deep generative model for graphs," CoRR, vol. abs/1802.08773, 2018. [Online]. Available: http://arxiv.org/abs/1802.08773

[51] K. Zhang, R. Chen, and H. Chen, "Numa-aware graph-structured analytics," SIGPLAN Not., vol. 50, no. 8, pp. 183–193, Jan. 2015. [Online]. Available: http://doi.acm.org/10.1145/2858788.2688507

[52] J. Zhong and B. He, "Medusa: Simplified graph processing on gpus," IEEE Transactions on Parallel and Distributed Systems, vol. 25, no. 6, pp. 1543–1552, June 2014.

[53] J. Zhou, G. Cui, Z. Zhang, C. Yang, Z. Liu, and M. Sun, "Graph neural networks: A review of methods and applications," ArXiv, vol. abs/1812.08434, 2018.

[54] S. Zhou and V. K. Prasanna, "Accelerating graph analytics on cpu-fpga heterogeneous platform," in 2017 29th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD), Oct 2017, pp. 137–144.

[55] R. Zhu, K. Zhao, H. Yang, W. Lin, C. Zhou, B. Ai, Y. Li, and J. Zhou, "Aligraph: A comprehensive graph neural network platform," CoRR, vol. abs/1902.08730, 2019. [Online]. Available: http://arxiv.org/abs/1902.08730

[56] X. Zhu, W. Chen, W. Zheng, and X. Ma, "Gemini: A computation-centric distributed graph processing system," in Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation, ser. OSDI'16. Berkeley, CA, USA: USENIX Association, 2016, pp. 301–316. [Online]. Available: http://dl.acm.org/citation.cfm?id=3026877.3026901

[57] X. Zhu, W. Han, and W. Chen, "Gridgraph: Large-scale graph processing on a single machine using 2-level hierarchical partitioning," in 2015 USENIX Annual Technical Conference (USENIX ATC 15). Santa Clara, CA: USENIX Association, Jul. 2015, pp. 375–386. [Online]. Available: https://www.usenix.org/conference/atc15/technical-session/presentation/zhu